

Práctica 2b: Broker de objetos

El objetivo de esta práctica es **desarrollar un sistema distribuido basado en Java RMI** (*Remote Method Invocation*) que actúe como broker de objetos, es decir, como intermediario entre clientes y servicios remotos. El broker debe encargarse de registrar, localizar y gestionar las invocaciones a objetos distribuidos de manera transparente para el cliente.

Esta práctica se plantea con diferentes niveles de dificultad, lo que permite a cada equipo adaptar el alcance de su desarrollo según su ritmo e intereses. Los niveles están pensados para evolucionar desde una arquitectura básica hasta una solución más completa y robusta. Cada equipo irá decidiendo hasta qué nivel desarrolla. **Esta práctica tendrá calificación numérica dentro de la asignatura.**

1. El patrón arquitectónico Broker

El patrón arquitectónico *Broker* se emplea en sistemas distribuidos para desacoplar a los clientes de los servidores que ofrecen servicios. En lugar de comunicarse directamente, los clientes interactúan con un componente intermedio denominado *broker* (ver Figura 1), que actúa como mediador entre ambas partes.

El broker es responsable de registrar los servidores disponibles, localizar dinámicamente el servicio adecuado para cada petición y gestionar la comunicación remota. De este modo, proporciona transparencia de ubicación: el cliente no necesita conocer dónde se encuentra físicamente el servidor ni cómo se realiza la comunicación subyacente.

Desde el punto de vista estructural, el sistema se compone de tres elementos principales: los clientes, que solicitan la ejecución de servicios; los servidores, que implementan la lógica de dichos servicios; y el broker, que coordina la interacción entre ambos. En muchas implementaciones aparecen además proxies o *stubs*, que permiten que las invocaciones remotas se realicen de forma similar a las locales.

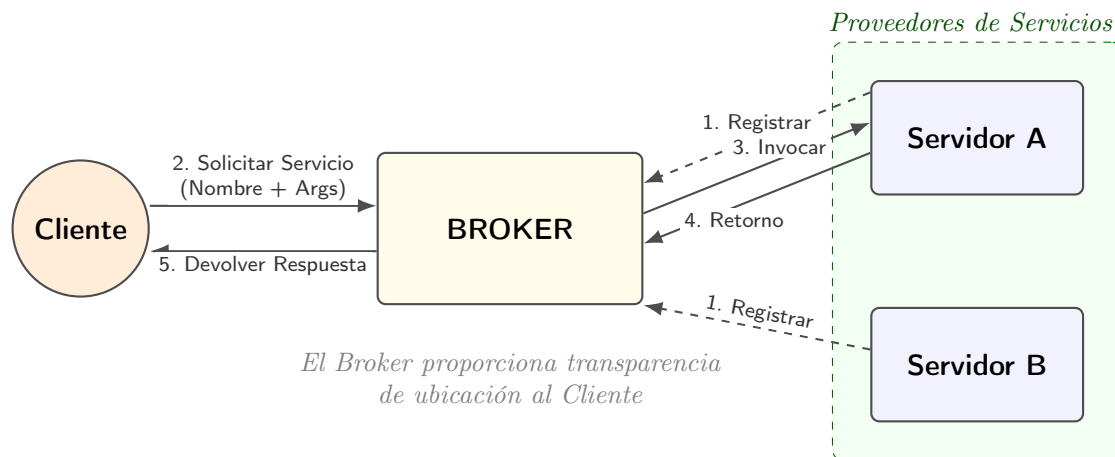


Figura 1: Arquitectura del patrón Broker y flujo de mensajes entre componentes.

El flujo de funcionamiento típico es el siguiente. En primer lugar, los servidores registran en el broker los servicios que ofrecen. Posteriormente, un cliente solicita la ejecución de un servicio indicando su nombre y los parámetros necesarios. El broker localiza el servidor correspondiente, realiza la invocación remota en nombre del cliente y finalmente devuelve el resultado obtenido.

En esta práctica, el broker será implementado por cada equipo utilizando Java RMI como mecanismo de comunicación remota entre componentes. RMI proporciona la infraestructura de invocación remota, mientras que la lógica de intermediación, registro y gestión de servicios será desarrollada explícitamente.

2. Componentes a implementar

Se implementarán **dos programas servidores independientes**, un broker central y uno o más clientes que realizarán peticiones a través del broker. A continuación, se detallan los requisitos específicos para cada componente.

2.1. Servidores

Cada equipo deberá implementar dos servidores que ofrezcan servicios remotos diferentes. Cada servidor definirá su propia interfaz remota (mediante `extends Remote`) y su correspondiente implementación. Además, cada servidor ofrecerá un conjunto distinto de métodos remotos.

Los servidores deberán registrarse en el broker. En la versión básica, el registro podrá consistir únicamente en comunicar su ubicación (nombre, IP y puerto). En versiones

avanzadas, el servidor también podrá registrar información adicional sobre los servicios ofrecidos, como el nombre del servicio, qué métodos ofrecen y cómo deben ser invocados.

En el nivel avanzado se requiere, además, un mayor desacoplamiento: los servidores no deben depender entre sí (no comparten interfaces comunes) ni depender funcionalmente del broker. En particular, únicamente deben conocer la dirección IP y el puerto del broker para enviar solicitudes de registro.

2.2. Broker

El broker actuará como intermediario entre clientes y servidores. Proporcionará una API remota que permitirá a los servidores registrarse y a los clientes solicitar la ejecución de servicios. Se plantean distintos niveles de implementación, organizados de forma progresiva.

2.2.1. Nivel básico

En el nivel básico, el mapeo entre los servicios disponibles y los servidores que los implementan estará definido en el código fuente del broker, por lo que añadir nuevos servicios requerirá recompilar el broker. Sin embargo, los servidores podrán estar distribuidos en distintas máquinas, y el broker deberá permitir registrar dinámicamente su ubicación.

El broker deberá implementar al menos los siguientes métodos:

```
// API para los servidores:
void registrar_servidor(String nombre_servidor,
                        String host_remoto_IP_puerto);

// API para los clientes:
Respuesta ejecutar_servicio(String nom_servicio,
                             List<Object> parametros_servicio);
```

- `registrar_servidor` permite a un servidor notificar al broker que está disponible e indicar su dirección.
- `ejecutar_servicio` permite a un cliente solicitar la ejecución de un servicio, que será resuelto por el broker mediante una invocación remota al servidor correspondiente.

2.2.2. Nivel avanzado

En el nivel avanzado, el broker permitirá el alta y baja dinámica de servicios, de manera que no será necesario recompilarlo cuando se añada o elimine un servicio. Para ello, el broker deberá mantener un registro actualizado de los servicios ofrecidos por cada servidor.

```
// API para los servidores:

// Registrar servidor (igual que en la versión básica)
void registrar_servidor(String nombre_servidor,
                        String host_remoto_IP_puerto);

// Registrar un nuevo servicio
void alta_servicio(String nombre_servidor,
                  String nom_servicio,
                  List<Object> lista_param,
                  String tipo_retorno);

// Dar de baja un servicio
void baja_servicio(String nombre_servidor,
                  String nom_servicio);

// API para los clientes:

// Listar servicios actualmente registrados
Servicios lista_servicios();

// Ejecutar un servicio de forma síncrona
Respuesta ejecutar_servicio(String nom_servicio,
                            List<Object> parametros_servicio);

// Ejecutar un servicio de forma asíncrona (nivel avanzado, versión
    ↪ n asíncrona)
void ejecutar_servicio_asinc(String nom_servicio,
                            List<Object> parametros_servicio);

// Obtener la respuesta de una ejecución asíncrona
Respuesta obtener_respuesta_asinc(String nom_servicio);
```

En esta práctica, **Respuesta** y **Servicios** deberán implementarse como clases serializables que encapsulen la información intercambiada entre cliente y broker.

2.2.3. Caso asíncrono

En la versión asíncrona, el broker deberá gestionar correctamente las solicitudes pendientes. En particular, deberá devolver un mensaje de error si el cliente no había realizado previamente la solicitud, si el cliente que pide la respuesta no coincide con el que realizó la petición, o si la respuesta ya fue entregada anteriormente.

Restricción: Para simplificar la lógica, un cliente no podrá ejecutar más de una vez el mismo servicio de forma asíncrona si no ha recogido antes la respuesta correspondiente.

2.3. Programa cliente

Se desarrollará una aplicación cliente que utilice los servicios ofrecidos por los servidores a través del Broker. El cliente implementará una lógica de aplicación libre, haciendo uso de los servicios remotos disponibles. Dependiendo del nivel de implementación alcanzado, podrá utilizar invocaciones síncronas y/o asíncronas.

3. Escenarios a defender

A continuación se describen los distintos **escenarios de ejecución** que cada equipo deberá preparar y demostrar durante la defensa del proyecto, en función del nivel de complejidad alcanzado.

3.1. Versiones básicas

1. El Broker y los servidores se registran en RMI y quedan a la espera de peticiones.
2. Los servidores se registran en el Broker.
3. Se ejecuta el cliente, que realiza una petición de servicio y muestra el resultado por pantalla.
4. Se cambia alguno de los servidores de máquina, pero no se recompila el Broker.
5. Se ejecuta nuevamente el cliente. El resultado se muestra correctamente, demostrando que el Broker funciona sin recompilación.

3.2. Versiones avanzadas

1. El Broker y los servidores se registran en RMI y quedan a la espera de peticiones.

2. Los servidores registran dinámicamente sus servicios en el Broker.
3. El cliente solicita al Broker la lista de servicios registrados.
4. El cliente muestra por pantalla la lista de servicios y el usuario selecciona uno de ellos.
5. El cliente solicita al Broker la ejecución del servicio seleccionado y muestra el resultado. A continuación, vuelve a mostrar el menú con la lista actual de servicios.
6. Uno de los servidores registra un nuevo servicio y da de baja otro sin recompilar el Broker.
7. El cliente vuelve a solicitar la lista de servicios y esta aparece actualizada, reflejando los cambios realizados.
8. El usuario selecciona y ejecuta otro servicio, y se muestra la respuesta correspondiente.

3.3. Versiones asíncronas

1. El cliente ejecuta una lógica de prueba que demuestra el funcionamiento correcto de la comunicación asíncrona.
2. El Broker gestiona correctamente los siguientes casos: (i) error si el cliente no había solicitado previamente el servicio; (ii) error si el cliente que solicita la respuesta no es el mismo que realizó la petición; (iii) error si la respuesta ya fue entregada anteriormente.
3. Se demuestra que el cliente no puede volver a solicitar el mismo servicio sin haber recogido antes la respuesta correspondiente.

4. Recomendaciones

1. Utilizad las máquinas del laboratorio **L1.02**. Al estar todas en la misma subred, se evitan configuraciones avanzadas de RMI. Las direcciones IP de estas máquinas van desde 155.210.154.191 hasta 155.210.154.210.
2. Podéis encender las máquinas del L1.02 desde `central.cps.unizar.es` utilizando el siguiente comando:

```
/usr/local/etc/wake -y lab102-210
```

Este comando encenderá la máquina cuya IP termina en .210. Para apagarla, utilizad:

```
/usr/local/etc/shutdown.sh -y lab102-210
```

3. Utilizad al menos **tres máquinas**: una para el **Broker**, una para los **dos servidores** y otra para el **cliente**.
4. Tened en cuenta que rmiregistry es único por máquina y puerto. Antes de lanzarlo, comprobad si ya existe un proceso **rmiregistry** en ejecución (por ejemplo, mediante el comando **top**). Si al iniciarlo aparece un error, probablemente otro proceso esté utilizando ese puerto; en ese caso, probad con uno diferente.

Evitad nombres genéricos como "**Broker**" o "**serverA**" para registrar vuestros objetos, ya que pueden colisionar con los de otros grupos. Utilizad nombres únicos, por ejemplo incorporando los tres últimos dígitos de vuestro NIA ("**Broker545**", "**serverB545**", etc.).

Cuando finalicéis la sesión, recordad cerrar el proceso de **rmiregistry** mediante:

```
kill -9 $pid
```

5. Los servidores y el cliente deben conocer, antes de ser compilados, en qué máquina se encuentra el Broker.

5. Entrega final

1. **Código fuente documentado**, con comentarios adecuados que faciliten la comprensión del sistema.
2. **Documentación de la arquitectura**, incluyendo:
 - Vista de **módulos**.
 - Vista de **componentes y conectores (CyC)**.
 - Vista de **distribución**.

Extensión máxima: 2 páginas (una hoja a doble cara).